A Geometric Multigrid Preconditioner for Shifted Boundary Method

Michał Wichrowski[*], Ajay Ajith [†]

November 12, 2025

1 Introduction

MW: Introduction needs to be rewritten.

2 Method formulation

We consider the Poisson problem as a model problem:

$$-\Delta u = f \quad \text{in } \Omega, \tag{1}$$
$$u = g \quad \text{on } \Gamma, \tag{2}$$

where $\Omega \subset \mathbb{R}^d$ $(d = 2, 3)$ is a domain with boundary $\Gamma = \partial\Omega$ as depicted in Figure 1, f is a given source term, and g is the prescribed Dirichlet boundary condition.

To solve this problem numerically, we first formulate it in a weak sense. We seek a solution u in an appropriate function space, $H^1(\Omega)$, which consists of functions that are square-integrable and whose first derivatives are also square-integrable. Multiplying the equation by a test function $v \in H^1(\Omega)$ and integrating over Ω, we obtain:

$$\int_\Omega \nabla u \cdot \nabla v \, dx - \int_\Gamma (\nabla u \cdot \mathbf{n}) \, v \, ds = \int_\Omega f \, v \, dx.$$

We next introduce a triangulation $\mathcal{T}_h$ consisting of quadrilateral (2D) or hexahedral (3D) elements of size h, and define a finite element space $\mathbb{V}_h \subset H^1(\Omega)$ using Lagrange polynomial elements of degree p. In classical finite element methods, the mesh conforms to the boundary (unlike the background mesh approach illustrated in Figure 1), and test functions v typically vanish on Γ to strongly enforce the Dirichlet condition $u = g$. When function spaces do not necessarily satisfy essential boundary conditions strongly, boundary conditions must be enforced weakly through additional integral terms on Γ.

Nitsche's method provides a way to weakly impose Dirichlet boundary conditions within a variational formulation without requiring the function space to satisfy the boundary conditions. It modifies the bilinear form by adding terms on the boundary Γ. The standard Nitsche formulation for the Poisson problem with Dirichlet boundary conditions $u = g$ on Γ seeks $u_h \in V_h$ such that for all $v_h \in \mathbb{V}_h$:

$$\begin{aligned}\int_\Omega \nabla u_h \cdot \nabla v_h \, dx \quad - \int_\Gamma (\nabla u_h \cdot \mathbf{n}) v_h \, ds - \alpha \int_\Gamma (\nabla v_h \cdot \mathbf{n}) u_h \, ds + \int_\Gamma \sigma u_h v_h \, ds = \\ = \int_\Omega f v_h \, dx - \int_\Gamma (\nabla v_h \cdot \mathbf{n}) g \, ds + \int_\Gamma \sigma_\Gamma g v_h \, ds.\end{aligned}$$

[*] Heidelberg University, Germany, mt.wichrowsk@uw.edu.pl
[†] Heidelberg University, Germany

Here, σ_Γ is a penalty parameter, typically chosen as $\sigma_\Gamma = \mathcal{O}(p^2 h^{-1})$ for mesh size h, and $\mathbf{n}$ is the outward unit normal vector to Γ. The choice of parameter $\alpha = 1$ leads to a symmetric formulation, while $\alpha = -1$ results in a non-symmetric formulation in which the penalty term can be skipped [2]

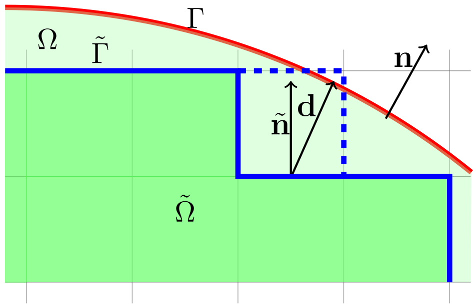

Figure 1: Schematic illustrating the background mesh, interior cells (green), the surrogate boundary $\tilde{\Gamma}$ (thick blue line) along the upper boundary of the interior cells, and the true boundary $\partial\Omega$ (red).

2.1 Shifted Boundary Method

In many applications, the domain Ω may have a complex geometry, making the generation of body-fitted meshes challenging. Non-body-fitted (unfitted) methods address this challenge by employing a background mesh $\mathcal{T}_h$ that does not conform to the boundary of Ω. The domain Ω is embedded within this background mesh, and cells of $\mathcal{T}_h$ are classified as active based on their intersection with Ω. The computational domain $\tilde{\Omega}$ is defined as the union of these active cells. In the original SBM formulation [5], only cells strictly contained in Ω were included, leading to the surrogate domain boundary depicted by the solid blue line in Figure 1. Consequently, the surrogate domain $\tilde{\Omega}$ is generally a subset of Ω, and its boundary $\tilde{\Gamma}$ does not coincide with the true boundary Γ.

Later extensions include intersected cells [7] with a volume fraction inside Ω greater than a threshold $\lambda \in [0, 1]$. In Figure 1, this corresponds to including additional cells as indicated by the dashed blue line. This approach reduces the distance between true and surrogate boundaries, but can lead to ill-conditioning.

To impose boundary conditions on $\tilde{\Gamma}$, the Dirichlet condition prescribed on the true boundary Γ is extrapolated to the surrogate boundary. This is typically accomplished by projecting points from $\tilde{\Gamma}$ onto the true boundary Γ along a suitable direction (e.g., the outward normal to $\tilde{\Gamma}$) and using a Taylor expansion to approximate the boundary values. This is accomplished via an *extension operator* $\mathcal{E}$, which maps functions defined on $\tilde{\Omega}$ to the surrogate boundary $\tilde{\Gamma}$.

We assume that the Dirichlet boundary condition g is given as a restriction of a function $u^\star$ defined on the entire domain Ω to the boundary Γ. While the choice of this function $u^\star$ (as an extension of g from Γ into Ω) is not unique, we take $u^\star$ to be equal to the solution u in the surrogate domain $\tilde{\Omega}$. For each point $\tilde{\mathbf{x}} \in \tilde{\Gamma}$, let $\mathbf{x} \in \Gamma$ be its closest point projection onto the true boundary, and let $\mathbf{d} = \mathbf{x} - \tilde{\mathbf{x}}$ be the shift vector. The function u is extended from Γ to $\tilde{\Gamma}$ using a Taylor expansion:

$$\mathcal{E}u^\star(\tilde{\mathbf{x}}) = u^\star(\mathbf{x}) - \mathbf{d} \cdot \nabla u(\tilde{\mathbf{x}}) + \cdots$$

where $\mathcal{E}u^\star$ denotes the extrapolated boundary condition. The function $u^\star$ is assumed to be smooth in a neighborhood of $\tilde{\Gamma}$, which allows for the Taylor expansion to be valid. By substituting it into the weak formulation, we obtain a variational formulation for the shifted boundary problem with the extrapolated boundary condition on $\tilde{\Gamma}$ enforced in a Nitsche-like manner.

The SBM weak formulation seeks $u_h \in V_h$ such that for all $v_h \in V_h$,

$$\int_{\tilde{\Omega}} \nabla u_h \cdot \nabla v_h \, dx - \int_{\tilde{\Gamma}} (\nabla u_h \cdot \tilde{\mathbf{n}}) v_h \, ds - \alpha \int_{\tilde{\Gamma}} (\nabla v_h \cdot \tilde{\mathbf{n}}) \, \mathcal{E} u_h \, ds + \int_{\tilde{\Gamma}} \sigma_\Gamma \, \mathcal{E} u_h \, v_h \, ds =$$
$$= \int_{\tilde{\Omega}} f v_h \, dx - \int_{\tilde{\Gamma}} (\nabla v_h \cdot \tilde{\mathbf{n}}) g^E \, ds + \int_{\tilde{\Gamma}} \sigma_\Gamma g v_h \, ds.$$

where $\tilde{\mathbf{n}}$ is the outward normal to $\tilde{\Gamma}$ and σ_Γ is a penalty parameter. We notice that our discrete solution u_h is a piecewise polynomial function defined on the background mesh $\mathcal{T}_h$, hence its Taylor expansion can be computed directly by evaluating the function values at the points on the true boundary Γ. This allows us to avoid computation of higher-order derivatives of u_h. Note that the resulting form is no longer symmetric due to the presence of the extrapolated boundary condition. We will refer to formulation with $\alpha = 1$ as the *quasi-symmetric* formulation.

Moreover, a simple 1D experiment using the domain $\Omega = [0, \xi]$ with a surrogate domain $\tilde{\Omega} = [0, 1]$ and a single cell shows that for Lagrange polynomials of degree $p > 1$, the stiffness matrix can have negative entries on the diagonal. As a result, the system matrix may lose diagonal dominance, which can adversely affect the stability and convergence of iterative solvers.

Notably, a penalty-free formulation ($\sigma_\Gamma = 0$) can be achieved by setting $\alpha = -1$, which alters the sign of the extrapolated term in the weak formulation [3]. This approach also results in a positive diagonal for the stiffness matrix, and both formulations yield optimal convergence rates. However, symmetry of the remaining terms in the weak formulation is not recovered. Consequently, the 1D system matrix for this problem can exhibit complex eigenvalues when the true boundary ξ is less than the surrogate boundary (i.e., $\xi < 1$). Figure **??** illustrates this by plotting the maximum imaginary part of the eigenvalues for this 1D problem across different polynomial degrees p.

Figure **??** reveals that complex eigenvalues with non-zero imaginary parts indeed arise for negative shifts (when the true domain boundary ξ is less than the surrogate domain boundary, $\xi < 1$). The penalty-free formulation (dashed lines) exhibits eigenvalues with larger imaginary parts compared to the quasi-symmetric penalty formulation (solid lines) in this regime, while for positive shifts ($\xi > 1$), the eigenvalues remain real for both formulations. The presence of complex eigenvalues is problematic for multigrid methods, as they can adversely affect smoothing properties and overall convergence of the preconditioner. Due to the tensor product structure of the Laplace operator, these 1D observations suggest that similar issues with complex eigenvalues may manifest in higher dimensions (2D and 3D), potentially compromising the effectiveness of standard smoothers and degrading multigrid performance.

Interestingly, for higher-order polynomials ($p > 1$), the penalty-free formulation can yield eigenvalues with nonzero imaginary parts even when the surrogate boundary coincides with the true boundary ($\xi = 1$). In contrast, the quasi-symmetric formulation is generally less prone to such imaginary eigenvalues. For this reason, we choose the quasi-symmetric formulation as the default in our work.

These observed spectral properties, particularly the emergence of complex eigenvalues, stem from the SBM's treatment of boundary conditions on the surrogate boundary $\tilde{\Gamma}$. The effects of this treatment are localized to the cells adjacent to $\tilde{\Gamma}$ where the extrapolation and non-standard penalty terms are applied. Intuitively, one might expect that the *Laplacian-like* properties are perturbed only within these boundary-adjacent cells. This localization suggests that it should be possible to resolve these local perturbations within the scope of individual cells. Discontinuous Galerkin (DG) methods offer a natural framework for such a cell-wise treatment, as they inherently allow for different solution behaviors and formulations on each element and provide mechanisms to couple them appropriately.

3 Multigrid Preconditioner

The DG-SBM formulation described above leads to a large, sparse linear system. Due to the lack of symmetry and possible indefiniteness of the resulting matrix, we employ a Krylov subspace method, specifically GMRES, for its solution. The convergence of GMRES is sensitive to the condition number of the system matrix that grows with both the number of elements in the mesh and the polynomial degree. Multigrid methods [4] rely on a hierarchy of meshes with varying resolution levels. Our Cartesian background mesh naturally facilitates the construction of such a nested hierarchy. We define a sequence of meshes $\{\mathcal{T}_\ell\}_{\ell=0}^{L}$, where ℓ denotes the level and L is the finest level. For each level, we identify active cells that have at least a fraction λ of their measure (area in 2D, volume in 3D) inside the domain. On each mesh $\mathcal{T}_\ell$, we define a finite element space $\mathbb{V}_\ell^{\mathrm{DG}}$ consisting of piecewise polynomials of degree p_ℓ. Since the set of active cells is determined by their intersection with the domain boundary, these sets are not necessarily nested between levels, which can affect the efficiency of standard inter-grid transfer operators. While our construction ensures a nested hierarchy of finite element spaces, the lack of a strict geometric relationship between active cells at different levels may reduce the effectiveness of transfer operators. We mitigate this potential efficiency loss by using only linear elements on geometrically coarser meshes and gradually increasing the polynomial degree only on the finest mesh. Hence the resulting total number of levels is $L + p$.

With this hierarchy of meshes and spaces established, the multigrid method is defined through two crucial components: the smoothers, which reduce high-frequency error components on each level, and the transfer operators, which move information between different resolution levels. These components, together with a coarse-grid solver, form the complete multigrid algorithm. For preconditioning, we use a single multigrid V-cycle.

3.1 Smoother

We first consider Richardson iterations with preconditioners P. Given the current approximation u_h and the right-hand side f_h, a smoothing step updates the approximation:

$$u_h^{k+1} = u_h^k + P(f_h - A_h u_h^k).$$

We decompose the space $\mathbb{V}_\ell^{\mathrm{DG}}$ into a sum of subspaces $\mathbb{V}_i$, i.e., $\mathbb{V}_\ell^{\mathrm{DG}} = \sum_{i=1}^{N} \mathbb{V}_i$. Let A_i be the restriction of A to the subspace $\mathbb{V}_i$. Then, the additive subspace correction preconditioner is defined as:

$$P = \omega \sum_{i=1}^{N} A_i^{-1}$$

where ω is a relaxation parameter. Alternatively, the successive subspace correction method is a subspace correction method where the subspaces $\mathbb{V}_i$ are visited in a sequential manner. The procedure can be performed by applying one Richardson iteration with the preconditioner P_i for each subspace $\mathbb{V}_i$. Since in each step only one subspace is corrected, the residual only changes locally and an efficient implementation is possible. The preconditioner updates the solution by sequentially applying corrections for each subspace. For each subspace $\mathbb{V}_i$, a correction is computed and applied:

$$u_h \leftarrow u_h + R_i^T A_i^{-1} R_i (f_h - A_h u_h).$$

This process is repeated for all subspaces $i = 1, \ldots, N$. If the subspaces are chosen as one-dimensional spaces spanned by the basis functions, then the preconditioner is equivalent to either Jacobi (additive) or Gauss-Seidel (successive).

In the context of DG, a natural choice for the subspaces $\mathbb{V}_i$ is the set of functions that are non-zero only on a single cell. This leads to a cell-wise smoother, where we correct the solution on each cell independently. This can be interpreted as a block Gauss-Seidel smoother, where each block corresponds to the degrees of freedom associated with a single cell.

MW: rewrite that part

Viewing the smoother as a subspace correction method reveals important insights into the convergence properties. The correction computed within a subspace $\mathbb{V}_i$ implicitly satisfies certain boundary conditions on the boundary of that subspace. This is particularly important in the context of the SBM, where the boundary conditions are shifted. A careless choice of subspace can lead to overconstraining the correction, preventing certain error components from being effectively smoothed. For instance, Jacobi and Gauss-Seidel smoothers for continuous finite element methods, which correspond to vertex-patch smoothing with zero correction at patch boundaries, conflict with the shifted boundary conditions and are not effective, as confirmed in our preliminary numerical experiments. In contrast, the cell-wise smoother aligns well with the DG method's element-based structure and takes into account the local nature of the SBM boundary conditions, correcting the solution on each cell independently, as illustrated in Figure 2.

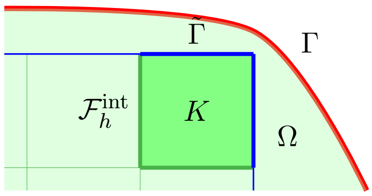

Figure 2: Illustration of a patch. Interior faces ($\mathcal{F}_h^{\text{int}}$, green) connect to neighboring active cells. For cells adjacent to the domain boundary, faces can be part of the surrogate boundary $\tilde{\Gamma}$ (blue), which is shifted from the true boundary Γ (red). The vertex-patch smoother solves should solve the local problem on patch P ...

3.2 Transfer Operators

We use the standard L^2 projection operators provided by `deal.II` for the prolongation ($I_{\ell-1}^{\ell}$) and restriction ($I_{\ell}^{\ell-1}$) operators, transferring data between coarser and finer grid levels. The restriction operator is the adjoint of the prolongation operator.

While these operators are standard in the context of multigrid methods, we note that the lack of a strict geometric relationship between active cells at different levels may reduce the effectiveness of transfer operators. We tried to reduce this potential efficiency loss by implementing a more sophisticated prolongation that would extrapolate the values that are not defined on the coarser mesh, and its adjoint as a restriction. However, this approach did not yield any significant improvements in the convergence rates. We therefore stick to the standard prolongation and restriction operators.

The choice of the threshold parameter λ directly influences the set of active cells, and thus the computational domain $\tilde{\Omega}_\ell$, on each multigrid level ℓ. A lower λ value generally results in more cells being classified as active, potentially leading to a surrogate domain that more closely envelops the true domain. However, the sets of active cells across different multigrid levels, $\tilde{\Omega}_\ell$ and $\tilde{\Omega}_{\ell-1}$, are not guaranteed to be nested. This lack of geometric nestedness can pose a challenge for standard transfer operators. For instance, a coarse cell might be active while some of its children on the finer level are not, or vice-versa. This mismatch can reduce the efficiency of inter-grid transfers, as information might be prolonged to or restricted from regions that are not consistently part of the active domain across levels. While our experiments with more sophisticated extrapolation-based transfer operators did not show significant gains, the interplay between λ, the resulting active cell configurations, and the transfer operator effectiveness remains an important consideration for multigrid performance in SBM.

4 Implementation details

The numerical implementation of the DG-SBM multigrid solver is based on the open-source finite element library `deal.II` [1]. It provides a comprehensive framework for the implementation of finite element methods, including mesh handling, finite element spaces, assembly of linear systems, and interfaces to various linear algebra solvers and preconditioners. Its flexibility and extensive capabilities make it well-suited for developing complex methods like the one presented here, which involves unfitted meshes and multigrid techniques.

The spatial discretization of the Poisson equation using the Discontinuous Galerkin method follows the standard framework available within `deal.II`. This involves defining the DG finite element space on the background Cartesian mesh, assembling the element-wise contributions to the global stiffness matrix and right-hand side vector, and handling the jump and average terms across interior faces as described in Section 2.2. The library's support for various DG formulations simplifies the implementation of the bilinear form $a_h(u_h, v_h)$.

4.1 Background mesh and geometry handling

When implementing SBM on a non-body-fitted mesh, we need to handle cells intersected by the true boundary Γ. We use a level set function $\phi(\mathbf{x})$ to implicitly define the domain $\Omega = \{\mathbf{x} \mid \phi(\mathbf{x}) < 0\}$, with $\Gamma = \{\mathbf{x} \mid \phi(\mathbf{x}) = 0\}$. Cells of the background mesh are classified based on their intersection with the zero level set: cells entirely inside Ω (interior), cells entirely outside Ω (exterior), and cells intersected by Γ. Then, for the intersected cells the fraction of the cell volume inside Ω is computed, and the cell is classified as active if this fraction is greater than a threshold λ. This is handled using non-matching quadrature rules implemented in `deal.II`, which are based on the techniques described in [6].

In our approach, degrees of freedom are formally assigned to all cells of the background mesh, including those that are classified as non-active. However, for non-active cells, the local contribution to the global system matrix is set to the identity matrix, and the corresponding entries in the right-hand side are set to zero. This effectively decouples the degrees of freedom in non-active cells from the system, ensuring that the solution is only computed within the computational domain $\tilde{\Omega}$ while simplifying the global matrix assembly process by maintaining a uniform structure of degrees of freedom (DoF) across all cells.

4.2 Processing surrogate boundary and matrix assemby

The SBM requires computing the closest point projection from points on the surrogate boundary $\tilde{\Gamma}$ to the true boundary Γ. For the specific test cases, where it is possible, this projection can be computed exactly. For general geometries, finding the closest point involves solving a local nonlinear optimization problem for each point on $\tilde{\Gamma}$. The problem is to find the point on the true boundary that minimizes the distance to a given point on the surrogate boundary, since the true boundary is given by the zero level set of the level set function $\phi(\mathbf{x})$, this can be formulated as:

$$\min_{\mathbf{x}\in\Gamma} \|\mathbf{x} - \tilde{\mathbf{x}}\|^2 \quad \text{s.t.} \quad \phi(\mathbf{x}) = 0.$$

We introduce a Lagrange multiplier μ to enforce the constraint $\phi(\mathbf{x}) = 0$. The Lagrangian function is:

$$\mathcal{L}(\mathbf{x}, \mu) = \|\mathbf{x} - \tilde{\mathbf{x}}\|^2 + \mu\phi(\mathbf{x}).$$

The necessary conditions for optimality are given by the stationary points of the Lagrangian, which satisfy the following system of equations:

$$\begin{aligned}
\nabla_{\mathbf{x}}\mathcal{L}(\mathbf{x}, \mu) &= 2(\mathbf{x} - \tilde{\mathbf{x}}) + \mu\nabla\phi(\mathbf{x}) = 0,\\
\frac{\partial \mathcal{L}}{\partial \mu}(\mathbf{x}, \mu) &= \phi(\mathbf{x}) = 0.
\end{aligned}$$

Solving this system yields the closest point $\mathbf{x}$ on the true boundary Γ to the point $\tilde{\mathbf{x}}$ on the surrogate boundary $\tilde{\Gamma}$. The nonlinear system is solved iteratively using a Newton-Raphson method. As the Newton-Raphson method requires the evaluation of the Jacobian matrix, we need to compute the second derivatives of the level set function $\phi(\mathbf{x})$. To ensure sufficient smoothness for derivative calculations in the closest point search, the level set function is represented using finite elements of degree 2 for $p = 1$ and degree p for $p > 1$.

The search for the closest point is performed only in the interior of the cells adjacent to the surrogate boundary. While this does not guarantee that the closest point is found inside the cell, it is expected that even if the closest point is outside the cell, the extrapolation will still yield a good approximation of the boundary condition. We will verify this assumption in the numerical results.

Finally, the extrapolation of the function values from the true boundary to the surrogate boundary required for the matrix assembly process is accomplished by evaluating the values of the basis functions at the points on the surrogate boundary. The assembly of the cell contributions and interior faces to the system matrix and right-hand side vector is performed using the standard finite element assembly process provided by `deal.II`.

4.3 Multigrid structures

For the multigrid hierarchy, we leverage `deal.II`'s built-in capabilities for handling nested meshes and defining finite element spaces on each level. The standard L^2 projection operators provided by `deal.II` are used for the prolongation ($I_{\ell-1}^{\ell}$) and restriction ($I_{\ell}^{\ell-1}$) operators, transferring data between coarser and finer grid levels.

The assembly of the system matrix A_ℓ on each level ℓ of the multigrid hierarchy follows a procedure analogous to that on the finest level. The level set function defining the domain geometry is interpolated onto the mesh $\mathcal{T}_\ell$. Based on this interpolated level set and the chosen threshold λ, active cells for level ℓ are identified. The DG-SBM bilinear form is then used to assemble the local contributions to A_ℓ only for these active cells. For cells deemed non-active on level ℓ, their corresponding entries in A_ℓ are set to effectively decouple them from the system by contributing an identity block to the matrix and zero to the right-hand side.

The smoother component of the multigrid V-cycle is implemented as a cell-wise Symmetric Successive Over-Relaxation (SSOR) method. The SSOR iteration proceeds by visiting each cell sequentially and solving a small local linear system corresponding to the block of degrees of freedom within that cell, using the most recently updated values from previously processed cells. This choice is motivated by its relative ease of tuning compared to other smoothers. For instance, a Block Jacobi smoother typically requires a carefully chosen relaxation parameter. In contrast, SSOR-based smoothers often perform well with a fixed relaxation parameter, frequently $\omega = 1$, reducing the need for problem-specific tuning. Unless stated otherwise, we use $\omega = 1$.

The current implementation focuses on testing the fundamental concepts and effectiveness of the multigrid preconditioner for the DG-SBM method. Therefore, it is based on serial computations using sparse matrices. While `deal.II` supports parallelization, the initial goal was to validate the multigrid algorithm's convergence properties and identify potential challenges in the DG-SBM context before scaling up to larger, parallel problems.

5 Numerical results

In this section, we present numerical results to evaluate the performance of the proposed DG-SBM multigrid preconditioner for the Poisson equation. We investigate its effectiveness in terms of convergence rates, iteration counts under mesh refinement (h-refinement) and polynomial degree increase (p-refinement). To better illustrate the multigrid preconditioner's performance characteristics and enable meaningful comparison of iteration counts, we solve the system to high accuracy with a tolerance of 10^{-12} for the relative residual reduction. This stringent tolerance results in higher iteration counts, making the differences in solver performance more apparent and facilitating clearer assessment of the multigrid effectiveness. Solver failure is defined as exceeding 100 GMRES iterations without reaching this tolerance. The initial background mesh is a square (or cube in 3D) domain covering the range $[-1.01, 1.01]$ in each dimension, subdivided into 4 cells in each coordinate direction. We use $\sigma_\Gamma = 5$ and $\sigma_F = 1$ for the penalty parameters in the SBM formulation. The interior penalty $\sigma_F = 1$ is chosen as a minimal value to ensure coercivity while minimizing its impact on solver convergence, whereas the boundary penalty $\sigma_\Gamma = 5$ is selected to be sufficiently large to ensure stability of the Nitsche coupling on the surrogate boundary.

The majority of our tests are conducted on a unit sphere (a unit disk in 2D, depicted on the left panel of Fig. **??**). While geometrically simple, the unit sphere serves as an insightful benchmark. Any sufficiently smooth (C^1) complex boundary, when viewed at a fine enough mesh resolution, locally resembles a flat plane. The unit sphere, due to its uniform curvature, presents a comprehensive range of intersection angles between the true boundary and the background mesh cells. Furthermore, the boundary intersects cells at various locations relative to cell centers and faces, leading to a diverse distribution of shift vector magnitudes and directions. This includes scenarios where the shift vector points from the surrogate boundary towards the interior of the true domain (which we denote as negative shifts if they oppose the outward normal of the surrogate boundary). The right panel of Figure **??** depicts the minimum and maximum shift magnitudes observed across the surrogate boundary for a typical discretization. The shift magnitudes are normalized by the cell size h; for a unit cell, the theoretically largest possible shift magnitude in 2D is $\sqrt{2}h$, occurring if the surrogate boundary point is at a cell corner and the true boundary passes through the diagonally opposite corner.

The primary metric for evaluating the multigrid preconditioner is the number of GMRES iterations required to reduce the initial residual by a factor of 10^{-12}. If the threshold is not met within 100 iterations, the solver is considered to have failed. All experiments are conducted using the implementation described in the previous section. As our solver depends on the value of the threshold parameter λ, we will first explore the solver performance on 2D problems as they are less computationally intensive and then show some results for 3D problems with tuned λ.

6 Conclusion

Acknowledgments

The author declares the use of language models (ChatGPT, Gemini, and Claude) to improve the clarity and readability of the manuscript. All scientific content and technical claims are solely the responsibility of the author.

The author would like to thank Guglielmo Scovazzi and Guido Kanschat for insightful discussions, and Luca Heltai for encouraging the exploration of this method.

References

[1] P. C. Africa, D. Arndt, W. Bangerth, B. Blais, M. Fehling, R. Gassmöller, T. Heister, L. Heltai, S. Kinnewig, M. Kronbichler, M. Maier, P. Munch, M. Schreter-Fleischhacker, J. P. Thiele, B. Turcksin, D. Wells, and V. Yushutin, *The deal.II library, Version 9.6*, Journal of Numerical Mathematics, 32 (2024), pp. 369–380.

[2] C. E. Baumann and J. T. Oden, *A discontinuous hp finite element method for convection—diffusion problems*, Computer Methods in Applied Mechanics and Engineering, 175 (1999), pp. 311–341.

[3] J. H. Collins, A. Lozinski, and G. Scovazzi, *A penalty-free shifted boundary method of arbitrary order*, Computer Methods in Applied Mechanics and Engineering, 417 (2023), p. 116301.

[4] W. Hackbusch and W. Hackbusch, *The Multi-Grid Method of the Second Kind*, Multi-Grid Methods and Applications, (1985), pp. 305–353.

[5] A. Main and G. Scovazzi, *The shifted boundary method for embedded domain computations. Part I: Poisson and Stokes problems*, Journal of Computational Physics, 372 (2018), pp. 972–995.

[6] R. I. Saye, *High-order quadrature methods for implicitly defined surfaces and volumes in hyperrectangles*, SIAM Journal on Scientific Computing, 37 (2015), pp. A993–A1019.

[7] C.-H. Yang, K. Saurabh, G. Scovazzi, C. Canuto, A. Krishnamurthy, and B. Ganapathysubramanian, *Optimal surrogate boundary selection and scalability studies for the shifted boundary method on octree meshes*, Computer Methods in Applied Mechanics and Engineering, 419 (2024), p. 116686.